
上海大学

SHANGHAI UNIVERSITY

2025-2026 春季学期

《自然语言语义处理概论》大作业

题目： 智能招聘匹配系统——基于 NLP 的职位推荐问答
助手

学 院	<u>计算机工程与科学学院</u>
	<u>计算机科学与技术</u>
专 业	<u>智能科学与技术</u>
学 号	<u>23122798, 23122734, 23122811,</u> <u>23122736</u>
学生姓名	<u>李泽勋, 夏颖, 叶佳颖, 丁佳悦</u>
提交日期	<u>2026/6/18</u>

目录

目录..... 1

1 系统概述..... 2

 1.1 研究背景与问题陈述..... 2

2 概要设计..... 3

 2.1 系统架构的逻辑分层..... 3

 2.2 数据处理流程设计..... 3

3 详细设计..... 4

 3.2 实体识别与规范化模块..... 5

 3.3 结构化画像抽取引擎..... 6

 3.4 语义匹配与加权排序算法..... 6

4 总结..... 8

 4.1 测试用例设计与执行结果..... 8

 4.2 结果深度分析与局限性探讨..... 9

 4.3 工作总结与收获..... 9

 4.4 未来改进方向..... 9

参考文献：..... 10

系统概述表

功能块个数：5 NLP 技术个数：5		
功能模块名称	模块概述	采用 NLP 相关技术列表
数据采集与标签推断	负责从招聘网站抓取原始职位数据，并通过预设的先验知识库对缺失标签的职位进行技术栈自动推断。	字符串匹配，规则推断
实体识别与规范化	识别用户输入中的技术术语（如缩写、全称），并利用自定义词典将其映射为标准化的表达形式，消除语义歧义。	命名实体识别，同义词映射/规范化
结构化画像抽取	从非结构化的自然语言对话中，精准提取工作年限、学历、薪资等关键数值指标，构建用户画像。	正则表达式，文本信息抽取
意图识别与分类	识别用户当前对话的真实意图（如职位搜索、薪资查询等），确保系统能够做出符合逻辑的交互响应。	意图分类
语义匹配与加权排序	将用户需求向量化，计算与职位库的语义相似度，并结合业务规则（技能重合、经验门槛等）进行多维度加权排序。	语句向量化，余弦相似度计算

评分表

得分项	得分
报告 20 分	
答辩 10 分	
功能得分 30 分	
技术得分 30 分	
整体评价 10 分	
总分	

1 系统概述

1.1 研究背景与问题陈述

在当今高度数字化的招聘市场中，求职者与职位之间的匹配效率是衡量人力资源配置水平的关键指标。然而，传统的招聘搜索模式主要依赖于基于倒排索引的关键词匹配。这种模式虽然在处理结构化查询时表现稳定，但在面对自然语言表达时存在显著的语义鸿沟。求职者往往倾向于使用口语化的描述来表达复杂的职业背景和期望，例如“具有 3 年 Java 开发经验并希望在北京寻找月薪 2 万以上职位”。对于此类描述，传统系统难以直接解析其中的工作年限、地域偏好及薪资梯度，导致搜索结果的精确度受限。

此外，技术术语的非规范化表达也是行业面临的长期痛点。在不同的语境下，缩写、全称及同义词（如“js”与“JavaScript”、“k8s”与“Kubernetes”）在字面上差异巨大，但在语义层面高度一致。如果系统缺乏对这些实体的规范化处理能力，将导致大量优质职位的漏报。因此，我们认为有必要开发一套基于自然语言处理技术的智能招聘匹配问答系统。通过引入深度语义理解和结构化信息抽取技术，我们期望打破传统搜索的字面局限，实现从“关键词检索”到“语义意图理解”的系统提升。

1.2 系统设计目标与核心价值

本项目的主要目标是构建一个能够深度理解用户求职意向并提供精准推荐的智能代理。系统设计的核心逻辑定位在三个层面：首先是提升语义理解的深度，利用预训练语言模型将文本映射到高维空间，捕捉深层的职业意图；其次是增强信息抽取的精度，从非结构化对话中识别技能、学历、年限等核心指标；最后是优化排序算法的业务逻辑贴合度，结合行业规则实现多维度的职位重排。

系统的核心价值在于其交互的灵活性和匹配的准确性。通过引入多轮对话机制，系统能够记忆用户的历史输入，在交互过程中逐步完善用户画像。这种渐进式需求挖掘的方式，不仅降低了用户的输入门槛，也让推荐结果能够随着对话的深入而愈发精准。我们期望通过本系统的实现，为求职者提供一种更直观、更高效的职位发现方式。

2 概要设计

2.1 系统架构的逻辑分层

为了实现复杂语义的处理与高效匹配，系统在架构上采用了分层设计模式，共分为四个核心层次：

1. 数据层：作为系统的基础，数据层负责存储原始的职位详情以及经过向量化处理的职位特征库。我们采用了 JSON 格式进行数据持久化，确保了数据结构的灵活性和读取的高效性。
2. NLP 处理层：这是系统的处理中枢，集成了从基础的分词、词性标注到高阶的向量编码、意图分类等一系列处理模块。该层利用 spaCy 和 SentenceTransformer 等成熟工具，为业务逻辑提供底层的语义支撑。
3. 业务逻辑层：负责协调各个 NLP 模块的调用顺序，并管理用户的会话状态。该层通过 Flask 框架实现，确保了前后端交互的稳定性和并发处理能力。
4. 交互层：提供简洁的 Web 对话界面。通过实时消息气泡展示，将后台复杂的处理逻辑转化为直观的用户交互体验。

2.2 数据处理流程设计

系统的数据处理流程遵循严格的流水线模式，确保了从原始网页数据到最终推荐结果的准确转化。

在离线阶段，我们首先运行 `scrape positions.py` 脚本。该脚本不仅负责网页结构的解析，还承担了初步的数据清洗任务。随后，`build_app_data.py` 脚本对抓取的数据进行二次加工。这一阶段的关键在于“JD 模板引擎”的应用，它将零散的职位描述重构为标准化的结构化文本，为后续的特征提取提供了高质量的语料。

在在线交互阶段，当系统接收到用户输入后，会触发级联的处理逻辑：首先是意图分类器识别用户目的；接着是 NER 模块提取技能实体；同时，画像抽取模块解析学历、经验等数值。系统将新提取的信息与 Session 中的历史信息合并，更新当前用户画像。最后，系统利用向量匹配算法在职位库中检索候选集，并结合加权打分逻辑输出最终推荐结果。

3 详细设计

本章将详细阐述系统中核心模块的设计思路，对项目的需求分析，工具选择，逻辑实现和代码解释分别进行说明。

3.1 职位数据采集与智能标签推断

在构建职位库的过程中，我们发现从招聘网站抓取的原始数据往往存在“标签缺失”或“描述不全”的问题。为了确保后续匹配算法有足够的特征锚点，我们需要一种方法为职位自动打上准确的技术标签。考虑到职位的行业属性较强，我们选择了基于规则的知识推断方法，并在 `scrape_positions.py` 中予以落实。

设计思路，我们建立了一个先验知识库 `SKILL INFERENCE`，通过识别职位名称（`Job Title`）中的行业关键词来反向推断其潜在的技术栈。这种方法虽然基于简单的规则匹配，但在 IT 领域具有较高的准确率和可解释性。

```
# scrape_positions.py 中的技能推断逻辑
SKILL_INFERENCE = {
    "后端": ["Java", "Python", "Go", "MySQL", "Redis", "Spring Boot"],
    "前端": ["HTML", "CSS", "JavaScript", "Vue.js", "React", "TypeScript"],
    "算法": ["Python", "PyTorch", "TensorFlow", "深度学习", "机器学习", "NLP"],
    "移动端": ["Android", "iOS", "Flutter", "Swift", "Kotlin"],
    "运维": ["Linux", "Docker", "Kubernetes", "Shell", "Nginx"]
}

def infer_skills(job_title):
    """
    根据职位标题推断潜在技能标签
    逻辑：遍历知识库键值，若标题包含键，则合并对应技能列表
    """
    inferred = set()
    for key, skills in SKILL_INFERENCE.items():
        if key in job_title:
            inferred.update(skills)
    return list(inferred)
```

在上述实现中，系统通过字符串匹配技术，实现了从简单的职位标题到丰富技术特征的自动扩展。这一预处理步骤为后续的语义召回奠定了坚实的数据基础。

3.2 实体识别与规范化模块

用户在对话中的表达具有高度的随意性，特别是在提及技术栈时，缩写（如“js”）、全称（如“JavaScript”）及混用现象非常普遍。为了确保系统能够识别这些非规范实体并实现语义对齐，我们需要在 `app_recruitment.py` 中设计一套鲁棒的实体规范化模块。我们选择了“统计模型+自定义词典”的混合策略，利用 `spaCy` 库进行基础 NLP 处理，并辅以自定义的映射表。

系统首先利用 `spaCy` 识别文本中的名词和专有名词，随后通过自定义的 `SKILL SYNONYMS` 字典进行标准化映射。

```
# app_recruitment.py 中的同义词规范化处理
SKILL_SYNONYMS = {
    'js': 'JavaScript',
    'vue': 'Vue.js',
    'k8s': 'Kubernetes',
    'py': 'Python',
    'reactjs': 'React',
    'node': 'Node.js',
    'golang': 'Go',
    'ts': 'TypeScript'
}

def normalize_skills(raw_skills):
    """
    对提取的原始技能进行规范化映射
    逻辑：统一转小写后查表替换，最后去重
    """
    normalized = []
    for skill in raw_skills:
        skill_lower = skill.lower()
        if skill_lower in SKILL_SYNONYMS:
            normalized.append(SKILL_SYNONYMS[skill_lower])
        else:
            # 对于不在字典中的词，保留原样（首字母大写处理）
            normalized.append(skill.capitalize() if len(skill) > 2 else
                               skill.upper())
    return list(set(normalized))
```

通过这一逻辑，系统能够将“js”准确映射为“JavaScript”。这种处理确保了不同表达方式在系统内部具有统一的特征表示，从而提升了匹配过程的鲁棒性，避免了因字面差异导致的语义偏离。

3.3 结构化画像抽取引擎

为了实现精准推荐，系统需要从自然语言中提取工作年限、学历、薪资等结构化指标。这些信息通常嵌入在复杂的句式里，直接提取难度较大。针对这一需求，我们设计了基于正则表达式的级联抽取引擎。我们选择正则表达式作为核心工具，是因为其在处理具有特定文本模式的数值提取任务时，具有极高的开发效率和执行准确率。

我们针对不同的画像维度设计了多组正则表达式。以工作经验提取为例，我们通过设置多个具有优先级的模式来覆盖不同的口语化表达。

```
# app_recruitment.py 中的画像抽取引擎
EXPERIENCE_PATTERNS = [
    r'(\d{1,2})\s*年\s*(?:工作)?经验',
    r'工作\s*(\d{1,2})\s*年',
    r'做了\s*(\d{1,2})\s*年',
    r'有\s*(\d{1,2})\s*年',
    r'经验\s*(\d{1,2})\s*年',
    r'^(\d{1,2})\s*年$'
]

def extract_experience(text):
    """
    从文本中提取工作年限数字
    逻辑：级联匹配多个正则模式，返回第一个匹配成功的数字
    """
    for pattern in EXPERIENCE_PATTERNS:
        match = re.search(pattern, text)
        if match:
            try:
                return int(match.group(1))
            except ValueError:
                continue
    return None
```

该模块能够从多种表达中准确剥离出年限数字。这种结构化数据的获取，为后续执行硬性门槛过滤（如“经验不低于 3 年”）和加权计算提供了可靠的数据支持。

3.4 语义匹配与加权排序算法

在召回职位后，如何根据用户需求的满足程度进行科学重排是系统的核心挑战。单一的语义相似度往往无法全面反映职位的实际匹配度。为此，我们设计了一套混合加权

排序算法。我们选择了 SentenceTransformer 模型作为语义召回的工具，并结合自定义的业务加权逻辑。

匹配过程分为两个阶段：首先利用向量余弦相似度进行初步召回，确保语义方向的一致性；随后引入技能重合度、职位名称匹配度及硬性条件达标情况作为权重因子进行重排。

```
# app_recruitment.py 中的核心打分算法
def calculate_match_score(user_profile, job_feature, semantic_sim):
    """
    综合语义相似度与业务规则的加权打分
    逻辑：基础分(语义) * 技能因子 * 职位因子 * 门槛因子
    """
    score = semantic_sim

    # 1. 技能重叠加成：每重合一个关键技能，得分提升 20%
    user_skills = set(user_profile.get('skills', []))
    job_skills = set(job_feature.get('skills', []))
    overlap_count = len(user_skills & job_skills)
    score *= (1 + 0.2 * overlap_count)

    # 2. 职位名称精准匹配加成：50%的巨额提振
    if user_profile.get('positions'):
        for pos in user_profile['positions']:
            if pos.lower() in job_feature['title'].lower():
                score *= 1.5
                break

    # 3. 经验门槛校验：满足要求则给予 30%的奖励分
    u_exp = user_profile.get('experience')
    j_exp = job_feature.get('experience_req')
    if u_exp is not None and j_exp is not None:
        if u_exp >= j_exp:
            score *= 1.3
        else:
            # 经验不足则进行惩罚
            score *= 0.7

    return round(score, 4)
```

在上述算法逻辑中，我们通过乘法因子对初始语义分进行修正。技能重叠和硬性条件的满足能够显著提升职位的最终得分。这种设计模拟了真实招聘场景中的筛选优先级，使得推荐结果在满足语义意图的同时，更符合实际的入职门槛要求。

4 总结

4.1 测试用例设计与执行结果

为了全面验证系统的功能表现，我们设计了涵盖不同交互场景的测试用例，并在系统上进行了实际运行。

用例编号	测试场景	用户输入	系统提取结果	推荐结果评价
T1	技能缩写识别	“我会 js 和 vue”	[JavaScript, Vue.js]	实体对齐准确
T2	画像信息抽取	“本科毕业，3 年经验”	学历:本科, 经验:3	结构化信息完整
T3	意图分类识别	“Python 拿多少钱？”	意图: salary_query	成功识别非求职意图
T4	模糊语义匹配	“想找做网页的工作”	推荐: 前端开发工程师	语义召回能力良好
T5	复杂需求组合	“北京 Java，20k 以上”	城市:北京, 技能:Java	满足多维度约束
T6	负向条件测试	“我不想要外包”	无法识别否定语义	表现欠佳，需改进
T7	跨轮次累积	1. “我会 Go” 2. “有 2 年经验”	画像: {Go, 2 年}	会话状态维护正常
T8	学历门槛测试	“高中学历找算法岗”	推荐得分显著降低	门槛过滤逻辑生效

示例输入：Python 拿多少钱

Python拿多少钱

我听到了：Python拿多少钱

识别意图：薪资咨询

识别到的技能/职位关键词：Python

为您匹配的职位：

1. Python开发工程师 ★★

薪资范围：14K-22K | 匹配度：100%

要求：本科及以上，2年以上Python开发经验...

2. 测试工程师 ★★

薪资范围：10K-18K | 匹配度：50%

要求：大专及以上，2年以上测试经验...

3. 算法工程师 ★★

薪资范围：25K-45K | 匹配度：39%

要求：硕士及以上，计算机/数学相关专业...

示例输入：北京 java，20k 以上



4.2 结果深度分析与局限性探讨

通过对 T4 用例的观察，我们发现系统在处理模糊需求时表现出了较强的语义理解能力。由于向量模型捕捉到了“网页”与“前端”在语义空间中的邻近性，系统能够跨越字面差异完成推荐。这验证了我们选择 SentenceTransformer 作为语义召回基石的正确性。

然而，在 T6 和 T8 等用例中也暴露出了一些局限性。目前的逻辑在处理否定语义（如“不考虑外包”）时表现欠佳，这反映了基于正则和简单向量匹配的局限性。此外，对于薪资区间的理解，目前的正则引擎还停留在单点数值提取阶段，缺乏对“区间覆盖”这一数学逻辑的深入处理。这些发现为我们后续的系统优化提供了明确的方向。

4.3 工作总结与收获

在本项目中，我们实现了一套完整的智能招聘匹配系统。通过集成实体识别、画像抽取及混合加权排序算法，系统能够处理自然语言形式的求职需求并提供相应的职位推荐。在开发过程中，我们深入探讨了如何将 NLP 技术应用于实际的业务场景，并验证了语义匹配与业务规则相结合的可行性。

我们认为，本项目的成功之处在于构建了一个闭环的处理 Pipeline：从原始数据的自动化采集与推断，到多维度的语义特征提取，再到模拟业务逻辑的加权排序。这一过程不仅加深了我们对 SentenceTransformer、spaCy 等工具的理解，也锻炼了我们解决非结构化数据处理问题的能力。

4.4 未来改进方向

尽管系统已经实现了预期的核心功能，但在智能化程度和交互深度上仍有提升空间。未来的研究工作可以从以下几个方面展开：

引入大规模预训练语言模型（LLM）：通过接入如 GPT-4 或 Qwen 等大模型，利用其强大的语义推理能力来解决否定语义识别和复杂逻辑解析问题。

构建职业技能知识图谱：通过建立技能间的关联（如“React”属于“前端框架”），实现更深层次的语义泛化，提升推荐的覆盖面。

优化对话引导机制：增加更自然的追问与信息补全引导，使系统能够更主动地挖掘用户的潜在需求。

通过这些改进，我们期望进一步提升系统的智能化水平，为用户提供更精准、更具人性化的智能招聘服务。

参考文献：

- [1] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
- [2] Mikolov, T., et al. (2013). Efficient Estimation of Word Representations in Vector Space.
- [3] Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings.
- [4] 周志华. 《机器学习》. 清华大学出版社.
- [5] Flask Documentation. Pallets Projects.